



Course Number : 0512.4490
Course Name : Advanced Computer Architecture Lab
Course Year: 2026

Lab #1: ASIC Hierarchical Verification: High Level Simulator

Student Name: Itay Zohar
Student ID: 209276633

Assignment 1 - Warm-up Questions

1. The command to load the constant (-171) to register 5 is -

```
asm_cmd(ADD, 5, 1, 0, -171); // R5 = imm(-171) + 0 = -171
```

2. In order to load memory address to register 6 we can use these commands -

```
asm_cmd(ADD, 6, 1, 0, 4000); // R6 = 4000 (address as a constant)
asm_cmd(LD, 6, 0, 6, 0); // R6 = MEM[R6] = MEM[4000]
```

3. In order to perform this operation we can use the following commands -

```
asm_cmd(SUB, 4, 3, 4, 0); // R4 = R3 - R4
asm_cmd(LSF, 5, 4, 1, 1); // R5 = R4 << 1 (multiply by 2)
```

4. For this operation we can use the following -

```
asm_cmd(ADD, 3, 1, 0, 7); // R3 = 7
asm_cmd(JLT, 0, 3, 1, target); // jump to target if R3(7) <
imm(target)
```

5. For this operation we can place the higher 16 bits with LHI, following by adding the lower 16 bits with ADD, i.e. -

```
asm_cmd(LHI, 2, 0, 0, 0xABCD); // R2[31:16] = 0xABCD (high 16 bits)
asm_cmd(ADD, 2, 2, 1, 0x1234); // R2[15:0] = 0x1234 (low 16 bits
via imm)
```

6. In order to perform subroutine calls, we can use JEQ command -

```
asm_cmd(JEQ, 0, 0, 0, subroutine_addr); // call: jump, R7 = return
address
// ... subroutine body ...
asm_cmd(JIN, 0, 7, 0, 0); // return: jump to address
in R7
```

However, this procedure will not work in the case of nested subroutine calls, since R7 might be overwritten by these calls.

In order to circumvent this issue, we can store the value of R7 with ST in a known memory location that is used explicitly for the stack, before calling the nested subroutine, and then load it back to R7 after the subroutine finishes.

As the depth and complexity of the nested subroutine increases, we might need to implement a solution to push and pop these memory addresses, but it isn't strictly necessary.

Assignment 2 - Example Program

1. This program computes a prefix sum, each output is the sum of all of the inputs up to its index, i.e. $\text{output}[0] = \text{input}[0]$, $\text{output}[1] = \text{input}[0] + \text{input}[1]$, $\text{output}[2] = \text{input}[0] + \text{input}[1] + \text{input}[2]$
2. The inputs are memory address 15-22 that contains the numbers 0-7 at the program initialization. R3 is used as the loop counter, and R4 is used as the loop limit.
3. The outputs are addresses 15-22, where each result is saved at the end of each loop iteration.
4. The assembly program can be understood as follows -

```
asm_cmd(ADD, 2, 1, 0, 15); // 0: R2 = 15
asm_cmd(ADD, 3, 1, 0, 1); // 1: R3 = 1
asm_cmd(ADD, 4, 1, 0, 8); // 2: R4 = 8
// LOOP:
asm_cmd(JEQ, 0, 3, 4, 11); // 3: if R3 == R4 jump to inst 11
asm_cmd(LD, 5, 0, 2, 0); // 4: R5 = MEM[R2]
asm_cmd(ADD, 2, 2, 1, 1); // 5: R2 = R2 + 1
asm_cmd(LD, 6, 0, 2, 0); // 6: R6 = MEM[R2]
asm_cmd(ADD, 6, 6, 5, 0); // 7: R6 = R6 + R5
asm_cmd(ST, 0, 6, 2, 0); // 8: MEM[R2] = R6
asm_cmd(ADD, 3, 3, 1, 1); // 9: R3 = R3 + 1
asm_cmd(JEQ, 0, 0, 0, 3); // 10: jump to LOOP
asm_cmd(HLT, 0, 0, 0, 0); // 11: HALT

for (i = 0; i < 8; i++) mem[15+i] = i; // Constants are planted into
the memory somewhere after the program code
```

5. This program contains 2 loads and 1 store each iteration, we can reduce 1 load -

```
asm_cmd(ADD, 2, 1, 0, 15); // 0: R2 = 15
asm_cmd(ADD, 3, 1, 0, 1); // 1: R3 = 1
asm_cmd(ADD, 4, 1, 0, 8); // 2: R4 = 8
asm_cmd(LD, 5, 0, 2, 0); // 3: R5 = MEM[15]
// LOOP:
asm_cmd(JEQ, 0, 3, 4, 10); // 4: if R3 == R4 → HALT
asm_cmd(ADD, 2, 2, 1, 1); // 5: R2 = R2 + 1
asm_cmd(LD, 6, 0, 2, 0); // 6: R6 = MEM[R2]
asm_cmd(ADD, 5, 5, 6, 0); // 7: R5 = R5 + R6
asm_cmd(ST, 0, 5, 2, 0); // 8: MEM[R2] = R5
asm_cmd(ADD, 3, 3, 1, 1); // 9: R3 = R3 + 1
asm_cmd(JEQ, 0, 0, 0, 4); // 10: jump to LOOP
asm_cmd(HLT, 0, 0, 0, 0); // 11: HALT
```

```
for (i = 0; i < 8; i++) mem[15+i] = i; // Constants are planted into  
the memory somewhere after the program code
```

In the revised example program, we use R5 to hold the cumulative sum. This means we only need to load the next element into R6, add it to R5, and store – saving one load per iteration, reducing memory accesses from 3 to 2 per iteration. It does however require loading R5 before we enter the loop, which means the actual saving here is the number of loop iterations - 1.

Assignment 4 - ISS Simulator Testing #2¹

```
/* INTEGER SQUARE ROOT -- Binary Search + Shift-and-Add Multiply
*
* ALGORITHM: Binary search on answer x in [lo, hi]
*   Each iteration: mid = (lo+hi)/2
*                   if mid*mid <= N: lo = mid+1
*                   else:           hi = mid
*   Until lo >= hi → answer = lo-1
*
* EFFICIENCY:
*   hi capped at 65535
*   Multiply unrolled over 16 bits: no loop overhead, 0(16) additions
*   Total: ~16 iterations × ~90 instructions = ~1440 instructions worst
case
*
* REGISTER MAP (main loop):
*   R2 = N (input, reloaded from MEM[1000] after multiply)
*   R3 = lo
*   R4 = hi
*   R5 = mid
*   R6 = mid*mid
*
* REGISTER MAP (during multiply):
*   R2 = AND scratch
*   R3 = bitmask
*   R4 = mid << i
*   R5 = mid
*   R6 = accumulator
*
* MEMORY:
*   MEM[1000] = N      (input)
*   MEM[1001] = result (output)
*   MEM[1002] = saved lo (during multiply)
*   MEM[1003] = saved hi (during multiply)
*/
```

¹ As confirmed by Shir, since this lab was conducted individually rather than in pairs, only one ISS simulator test was required. Accordingly, only Simulator Test 2 was performed, and Simulator Test 1 was not conducted.

```

// pc=0: Load N
asm_cmd(LD, 2, 0, 1, 1000);

// pc=1-3: Init bounds
asm_cmd(ADD, 3, 0, 0, 0); // R3 = lo = 0
asm_cmd(LHI, 4, 0, 1, 1); // R4 = 0x00010000
asm_cmd(SUB, 4, 4, 1, 1); // R4 = hi = 65535

// pc=4: Loop guard
asm_cmd(JLE, 0, 4, 3, 99);

// pc=5-7: mid = lo + ((hi - lo) >> 1)
asm_cmd(SUB, 5, 4, 3, 0); // R5 = hi - lo
asm_cmd(RSF, 5, 5, 1, 1); // R5 = (hi - lo) >> 1
asm_cmd(ADD, 5, 5, 3, 0); // R5 = mid

// pc=8-9: Save lo/hi before multiply clobbers R3/R4
asm_cmd(ST, 0, 3, 1, 1002);
asm_cmd(ST, 0, 4, 1, 1003);

// pc=10-12: Multiply init (compute mid*mid in R6)
asm_cmd(ADD, 6, 0, 0, 0); // R6 = acc = 0
asm_cmd(ADD, 3, 0, 1, 1); // R3 = mask = 1
asm_cmd(ADD, 4, 5, 0, 0); // R4 = shifted_mid = mid

/* bit 0 (pc 13-17) */
asm_cmd(AND, 2, 5, 3, 0); // R2 = mid & mask -- isolate bit 0 of mid
asm_cmd(JEQ, 0, 2, 0, 16); // if bit 0 == 0, skip accumulate (jump to
pc=16)
asm_cmd(ADD, 6, 6, 4, 0); // acc += shifted_mid (mid << 0)
asm_cmd(LSF, 3, 3, 1, 1); // mask <= 1 (now selects bit 1)
asm_cmd(LSF, 4, 4, 1, 1); // shifted_mid <= 1 (now holds mid << 1)

/* bit 1 (pc 18-22) */
asm_cmd(AND, 2, 5, 3, 0); // R2 = mid & mask -- isolate bit 1 of mid
asm_cmd(JEQ, 0, 2, 0, 21); // if bit 1 == 0, skip accumulate (jump to
pc=21)
asm_cmd(ADD, 6, 6, 4, 0); // acc += shifted_mid (mid << 1)
asm_cmd(LSF, 3, 3, 1, 1); // mask <= 1 (now selects bit 2)
asm_cmd(LSF, 4, 4, 1, 1); // shifted_mid <= 1 (now holds mid << 2)

/* bit 2 (pc 23-27) */
asm_cmd(AND, 2, 5, 3, 0); // R2 = mid & mask -- isolate bit 2 of mid

```

```

asm_cmd(JEQ, 0, 2, 0, 26);    // if bit 2 == 0, skip accumulate (jump to
pc=26)
asm_cmd(ADD, 6, 6, 4, 0);    // acc += shifted_mid (mid << 2)
asm_cmd(LSF, 3, 3, 1, 1);    // mask <=< 1 (now selects bit 3)
asm_cmd(LSF, 4, 4, 1, 1);    // shifted_mid <=< 1 (now holds mid << 3)

/* bit 3 (pc 28-32) */
asm_cmd(AND, 2, 5, 3, 0);    // R2 = mid & mask -- isolate bit 3 of mid
asm_cmd(JEQ, 0, 2, 0, 31);    // if bit 3 == 0, skip accumulate (jump to
pc=31)
asm_cmd(ADD, 6, 6, 4, 0);    // acc += shifted_mid (mid << 3)
asm_cmd(LSF, 3, 3, 1, 1);    // mask <=< 1 (now selects bit 4)
asm_cmd(LSF, 4, 4, 1, 1);    // shifted_mid <=< 1 (now holds mid << 4)

/* bit 4 (pc 33-37) */
asm_cmd(AND, 2, 5, 3, 0);    // R2 = mid & mask -- isolate bit 4 of mid
asm_cmd(JEQ, 0, 2, 0, 36);    // if bit 4 == 0, skip accumulate (jump to
pc=36)
asm_cmd(ADD, 6, 6, 4, 0);    // acc += shifted_mid (mid << 4)
asm_cmd(LSF, 3, 3, 1, 1);    // mask <=< 1 (now selects bit 5)
asm_cmd(LSF, 4, 4, 1, 1);    // shifted_mid <=< 1 (now holds mid << 5)

/* bit 5 (pc 38-42) */
asm_cmd(AND, 2, 5, 3, 0);    // R2 = mid & mask -- isolate bit 5 of mid
asm_cmd(JEQ, 0, 2, 0, 41);    // if bit 5 == 0, skip accumulate (jump to
pc=41)
asm_cmd(ADD, 6, 6, 4, 0);    // acc += shifted_mid (mid << 5)
asm_cmd(LSF, 3, 3, 1, 1);    // mask <=< 1 (now selects bit 6)
asm_cmd(LSF, 4, 4, 1, 1);    // shifted_mid <=< 1 (now holds mid << 6)

/* bit 6 (pc 43-47) */
asm_cmd(AND, 2, 5, 3, 0);    // R2 = mid & mask -- isolate bit 6 of mid
asm_cmd(JEQ, 0, 2, 0, 46);    // if bit 6 == 0, skip accumulate (jump to
pc=46)
asm_cmd(ADD, 6, 6, 4, 0);    // acc += shifted_mid (mid << 6)
asm_cmd(LSF, 3, 3, 1, 1);    // mask <=< 1 (now selects bit 7)
asm_cmd(LSF, 4, 4, 1, 1);    // shifted_mid <=< 1 (now holds mid << 7)

/* bit 7 (pc 48-52) */
asm_cmd(AND, 2, 5, 3, 0);    // R2 = mid & mask -- isolate bit 7 of mid
asm_cmd(JEQ, 0, 2, 0, 51);    // if bit 7 == 0, skip accumulate (jump to
pc=51)
asm_cmd(ADD, 6, 6, 4, 0);    // acc += shifted_mid (mid << 7)

```

```

asm_cmd(LSF, 3, 3, 1, 1);      // mask <= 1  (now selects bit 8)
asm_cmd(LSF, 4, 4, 1, 1);      // shifted_mid <= 1  (now holds mid << 8)

/* bit 8  (pc 53-57) */
asm_cmd(AND, 2, 5, 3, 0);      // R2 = mid & mask  -- isolate bit 8 of mid
asm_cmd(JEQ, 0, 2, 0, 56);      // if bit 8 == 0, skip accumulate (jump to
pc=56)
asm_cmd(ADD, 6, 6, 4, 0);      // acc += shifted_mid  (mid << 8)
asm_cmd(LSF, 3, 3, 1, 1);      // mask <= 1  (now selects bit 9)
asm_cmd(LSF, 4, 4, 1, 1);      // shifted_mid <= 1  (now holds mid << 9)

/* bit 9  (pc 58-62) */
asm_cmd(AND, 2, 5, 3, 0);      // R2 = mid & mask  -- isolate bit 9 of mid
asm_cmd(JEQ, 0, 2, 0, 61);      // if bit 9 == 0, skip accumulate (jump to
pc=61)
asm_cmd(ADD, 6, 6, 4, 0);      // acc += shifted_mid  (mid << 9)
asm_cmd(LSF, 3, 3, 1, 1);      // mask <= 1  (now selects bit 10)
asm_cmd(LSF, 4, 4, 1, 1);      // shifted_mid <= 1  (now holds mid << 10)

/* bit 10 (pc 63-67) */
asm_cmd(AND, 2, 5, 3, 0);      // R2 = mid & mask  -- isolate bit 10 of
mid
asm_cmd(JEQ, 0, 2, 0, 66);      // if bit 10 == 0, skip accumulate (jump to
pc=66)
asm_cmd(ADD, 6, 6, 4, 0);      // acc += shifted_mid  (mid << 10)
asm_cmd(LSF, 3, 3, 1, 1);      // mask <= 1  (now selects bit 11)
asm_cmd(LSF, 4, 4, 1, 1);      // shifted_mid <= 1  (now holds mid << 11)

/* bit 11 (pc 68-72) */
asm_cmd(AND, 2, 5, 3, 0);      // R2 = mid & mask  -- isolate bit 11 of
mid
asm_cmd(JEQ, 0, 2, 0, 71);      // if bit 11 == 0, skip accumulate (jump to
pc=71)
asm_cmd(ADD, 6, 6, 4, 0);      // acc += shifted_mid  (mid << 11)
asm_cmd(LSF, 3, 3, 1, 1);      // mask <= 1  (now selects bit 12)
asm_cmd(LSF, 4, 4, 1, 1);      // shifted_mid <= 1  (now holds mid << 12)

/* bit 12 (pc 73-77) */
asm_cmd(AND, 2, 5, 3, 0);      // R2 = mid & mask  -- isolate bit 12 of
mid
asm_cmd(JEQ, 0, 2, 0, 76);      // if bit 12 == 0, skip accumulate (jump to
pc=76)
asm_cmd(ADD, 6, 6, 4, 0);      // acc += shifted_mid  (mid << 12)

```

```

asm_cmd(LSF, 3, 3, 1, 1);      // mask <= 1  (now selects bit 13)
asm_cmd(LSF, 4, 4, 1, 1);      // shifted_mid <= 1  (now holds mid << 13)

/* bit 13 (pc 78-82) */
asm_cmd(AND, 2, 5, 3, 0);      // R2 = mid & mask  -- isolate bit 13 of
mid
asm_cmd(JEQ, 0, 2, 0, 81);      // if bit 13 == 0, skip accumulate (jump to
pc=81)
asm_cmd(ADD, 6, 6, 4, 0);      // acc += shifted_mid  (mid << 13)
asm_cmd(LSF, 3, 3, 1, 1);      // mask <= 1  (now selects bit 14)
asm_cmd(LSF, 4, 4, 1, 1);      // shifted_mid <= 1  (now holds mid << 14)

/* bit 14 (pc 83-87) */
asm_cmd(AND, 2, 5, 3, 0);      // R2 = mid & mask  -- isolate bit 14 of
mid
asm_cmd(JEQ, 0, 2, 0, 86);      // if bit 14 == 0, skip accumulate (jump to
pc=86)
asm_cmd(ADD, 6, 6, 4, 0);      // acc += shifted_mid  (mid << 14)
asm_cmd(LSF, 3, 3, 1, 1);      // mask <= 1  (now selects bit 15)
asm_cmd(LSF, 4, 4, 1, 1);      // shifted_mid <= 1  (now holds mid << 15)

/* bit 15 (pc 88-90): last bit -- no LSFs needed after */
asm_cmd(AND, 2, 5, 3, 0);      // R2 = mid & mask  -- isolate bit 15 of
mid
asm_cmd(JEQ, 0, 2, 0, 91);      // if bit 15 == 0, skip accumulate (jump to
restore block)
asm_cmd(ADD, 6, 6, 4, 0);      // acc += shifted_mid  (mid << 15) -- R6 =
mid*mid complete

// pc=91-93: Restore N, lo, hi (clobbered during multiply)
asm_cmd(LD, 2, 0, 1, 1000);     // R2 = N  (reload from MEM[1000])
asm_cmd(LD, 3, 0, 1, 1002);     // R3 = lo (reload from MEM[1002])
asm_cmd(LD, 4, 0, 1, 1003);     // R4 = hi (reload from MEM[1003])

// pc=94: if N < mid*mid, jump to hi=mid at pc=97
asm_cmd(JLT, 0, 2, 6, 97);      // N < mid^2  => hi = mid
// pc=95: lo = mid + 1  (mid^2 <= N)
asm_cmd(ADD, 3, 5, 1, 1);
// pc=96: loop back
asm_cmd(JEQ, 0, 0, 0, 4);
// pc=97: hi = mid  (mid^2 > N)
asm_cmd(ADD, 4, 5, 0, 0);
// pc=98: loop back

```

```
asm_cmd(JEQ, 0, 0, 0, 4);

// pc=99-101: Finish
asm_cmd(SUB, 3, 3, 1, 1);    // R3 = lo - 1 = floor(sqrt(N))
asm_cmd(ST, 0, 3, 1, 1001); // MEM[1001] = result
asm_cmd(HLT, 0, 0, 0, 0);

mem[1000] = 4937775;        // input N (sqrt ≈ 2222)
```

Assignment 5 - ISS Simulator Implementation

Attached to this document is the ISS simulator, iss.c. This program can be compiled to an executable, using the makefile as instructed.